Why a SOC feeds its own CSV or PCAP traffic to an LLM pipeline, and how AgentMeter helps

Project	AgentMeter: Benchmarking Open-Source LLM Resource Efficiency in Agentic Network Threat Detection Pipelines
Purpose	Answer to the supervisor: the full process, from the SOC's problem to its deployment decision
Date	November 2026

1. Summary

> A Security Operations Centre (SOC) receives more network alerts than its analysts can examine, so it wants LLM agents to triage each flow. Its traffic is confidential and cannot be sent to cloud AI services, so the agents must run on open-source models on the SOC's own, limited GPU. Before deploying one, the SOC must know which model is fast enough to keep up with its alert volume, fits its GPU, costs least, and still gives usable answers.
>
> The SOC uploads its own CSV or PCAP because the cost of running the agents depends on the traffic they read. AgentMeter runs the real agentic threat-detection pipeline (Perceive, Reason, Decide, Act) on that traffic, with every model answering exactly the same flows, and measures latency, GPU memory, tokens, energy and cost while checking every verdict. The SOC gets a ranked, like-for-like comparison and can choose the model it can actually deploy.

2. The SOC's problem

1. **Too many alerts, too few analysts.** A SOC sees thousands of flows and alerts a day. Alerts left in a queue are attacks nobody has examined yet, which lengthens the mean time to detect and respond (MTTD, MTTR) and leads to missed incidents.
2. **It wants LLM agents to triage each flow.** For every flow, agents read the features, explain what is happening, assign a label and a MITRE ATT&CK technique, and recommend an action. This is the first-line triage an analyst would otherwise do by hand.
3. **The traffic cannot leave its network.** Flows contain internal addresses, user activity and sometimes credentials. Sending them to a cloud LLM API breaks confidentiality and, often, policy or law. The agents must therefore run on open-source models, on the SOC's own GPU, quantised to 4-bit so that they fit.
4. **So it must choose a model.** Open-source models differ widely in speed, memory and cost. A model that is too slow builds a backlog; one that does not fit in GPU memory cannot be deployed; one that is expensive takes budget from other defences.

3. Why the SOC uploads its own CSV or PCAP

Cost depends on the input
Each flow becomes a prompt for four agents in a row. The flow's features set the prompt length, hand-offs between agents add more, and answers vary. Different traffic gives different latency, memory and tokens.

Results must describe this SOC
A model that is fast on short chat questions may be slow on network flows. Only the SOC's own traffic shows whether a model keeps up with its alert volume on its GPU.

Same flows for every model
The prepared set is fingerprinted (SHA-256), so every model answers exactly the same flows on the same GPU and settings. Any difference belongs to the model.

INPUT	WHERE THE SOC GETS IT	WHAT AGENTMETER DOES WITH IT	EXTRA IT ALLOWS
PCAP (raw packets)	A network sensor or tap, tcpdump, Wireshark	Converts packets into flows (CICFlowMeter features), samples N flows	Efficiency results on real, unlabelled traffic
CSV (flows)	A CICFlowMeter export, or a public dataset such as CIC-IDS2017	Validates the columns, samples N flows	If labelled, the agents' answers are also checked against the labels (reported as context, not scored)

4. The end-to-end process

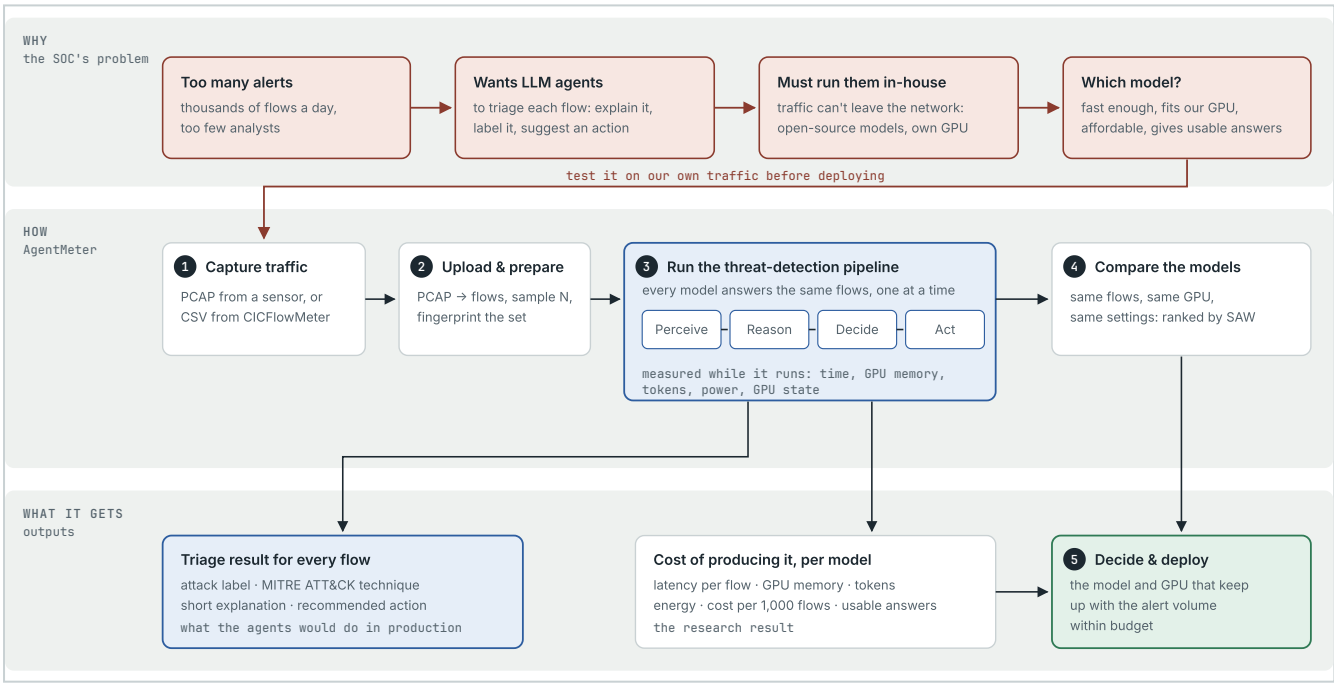


Figure 1. Top: the SOC's problem. Middle: what AgentMeter does. Bottom: what the SOC takes away. The uploaded traffic is the workload; the pipeline performs real triage on it, and AgentMeter measures what that triage costs for each model.

1. **Capture traffic.** The SOC exports a PCAP from a sensor, or a flow CSV from CICFlowMeter.
2. **Upload and prepare.** AgentMeter turns packets into flows, samples the number of flows requested, and fingerprints the prepared set. Preparation runs on the GPU server, or in the browser while the GPU is busy; both use the same code, so the flows and fingerprint are identical.
3. **Run the threat-detection pipeline.** Each selected model (one to four, 4-bit NF4, one at a time on the GPU) passes every flow through Perceive, Reason, Decide and Act. While it runs, AgentMeter

records time per flow and per agent, GPU memory, tokens, board power and GPU state.

4. **Compare the models.** Results are scored and ranked with Simple Additive Weighting (SAW) on latency, VRAM and tokens. Only sessions on the same flows, GPU and settings are compared; repeated runs show whether results are stable.
5. **Decide and deploy.** The SOC reads the session tabs and the PDF report, checks each model against its own limits, and deploys the model and GPU that keep up with its alert volume within budget.

5. How AgentMeter solves it

Each security need of the SOC becomes a resource-efficiency requirement, and AgentMeter measures whether a model meets it.

```
Security requirement → resource constraint → AgentMeter measurement → deployment decision
```

SOC NEED	WHY IT MATTERS FOR SECURITY	EFFICIENCY QUESTION	WHAT AGENTMETER MEASURES
Keep up with the alert volume	A backlog delays detection and response	Can the model triage flows as fast as they arrive?	Latency per flow, p95, throughput, queue simulation
Triage every flow with LLM agents	Broken or missing verdicts are as dangerous as none	What does four-agent triage cost, and are answers usable?	Tokens per flow, hand-off overhead, usable answers, answer consistency
Keep traffic inside the network	Confidentiality rules out cloud LLM APIs	Does an open-source model fit the SOC's GPU, at what cost?	Working and total VRAM (4-bit NF4), energy per flow, cost per 1,000 flows
Choose the right model	A wrong choice is unsafe (backlog) or wastes budget	Which model is most efficient with usable output?	SAW ranking, like-for-like Compare, fit to the SOC's limits, Repeatability

What the SOC receives

- **Per flow:** the agents' label, MITRE ATT&CK technique, explanation and recommended action, showing what the pipeline would do in production.
- **Per model:** latency (mean, p95, p99), throughput, working and total GPU memory, tokens, energy and cost per 1,000 flows, share of usable answers, GPU state during the run, and the SAW score.
- **Across models:** a ranking on the same flows, a check against the SOC's own limits, and stability over repeated runs, together with a PDF report and a reproducibility package for each session.

6. Worked example (illustrative figures)

A SOC sees 50,000 flows a day and has one 24 GB GPU rented at $0.32 per hour. It benchmarks two models on a sample of its traffic.

MODEL	SECONDS / FLOW	HOURS / 50,000 FLOWS	WORKING VRAM	COST / 1,000 FLOWS	VERDICT
Model A	2.1	29.2	9.8 GB	$0.19	Cannot keep up: a day of traffic takes more than a day
Model B	0.9	12.5	6.1 GB	$0.08	Keeps up with room to spare, at under half the cost

Hours = seconds per flow × 50,000 ÷ 3,600. Cost per 1,000 flows = seconds per flow × 1,000 ÷ 3,600 × hourly GPU price. AgentMeter computes both from the measured latency and the GPU's real price.

7. Scope

- AgentMeter is a resource-efficiency benchmark. It does not propose a new detection method; the threat-detection pipeline is the realistic workload it measures.
- Detection quality is reported as context: every verdict is checked for usability, and agreement with labels is shown when the CSV is labelled, but it is not part of the efficiency score.
- Results are compared only on the same prepared set, GPU and settings. Comparing different GPUs is future work.

8. Conclusion

> In a SOC, resource efficiency is a security property. Confidentiality forces LLM agents onto open-source models on the SOC's own limited GPUs, and alert volume demands that they triage fast enough to keep detection timely. A SOC therefore uploads its own CSV or PCAP so that AgentMeter can run the real agentic threat-detection pipeline on that traffic and measure what each model costs to triage it, while checking that every verdict is usable. The result tells the SOC which model it can actually deploy.